“Langchain Notes (1)”

Langchain has quickly become one of the most powerful frameworks for building applications with large language models (LLMs). Whether you’re just starting out or looking to level up your skills, this guide takes you through the fundamentals all the way to advanced concepts — step by step.

What is Langchain ?

Langchain is an open-source framework for developing applications powered by large language models (LLMs).

Why do we need Langchain ?

Imagine you’re developing an app that lets users upload PDFs of machine learning and interact with them conversationally. Users might ask:

📌 “Summarize page five in simple terms.”

📌 “Generate true or false questions on linear regression.”

📌 “Provide concise notes on decision trees.”

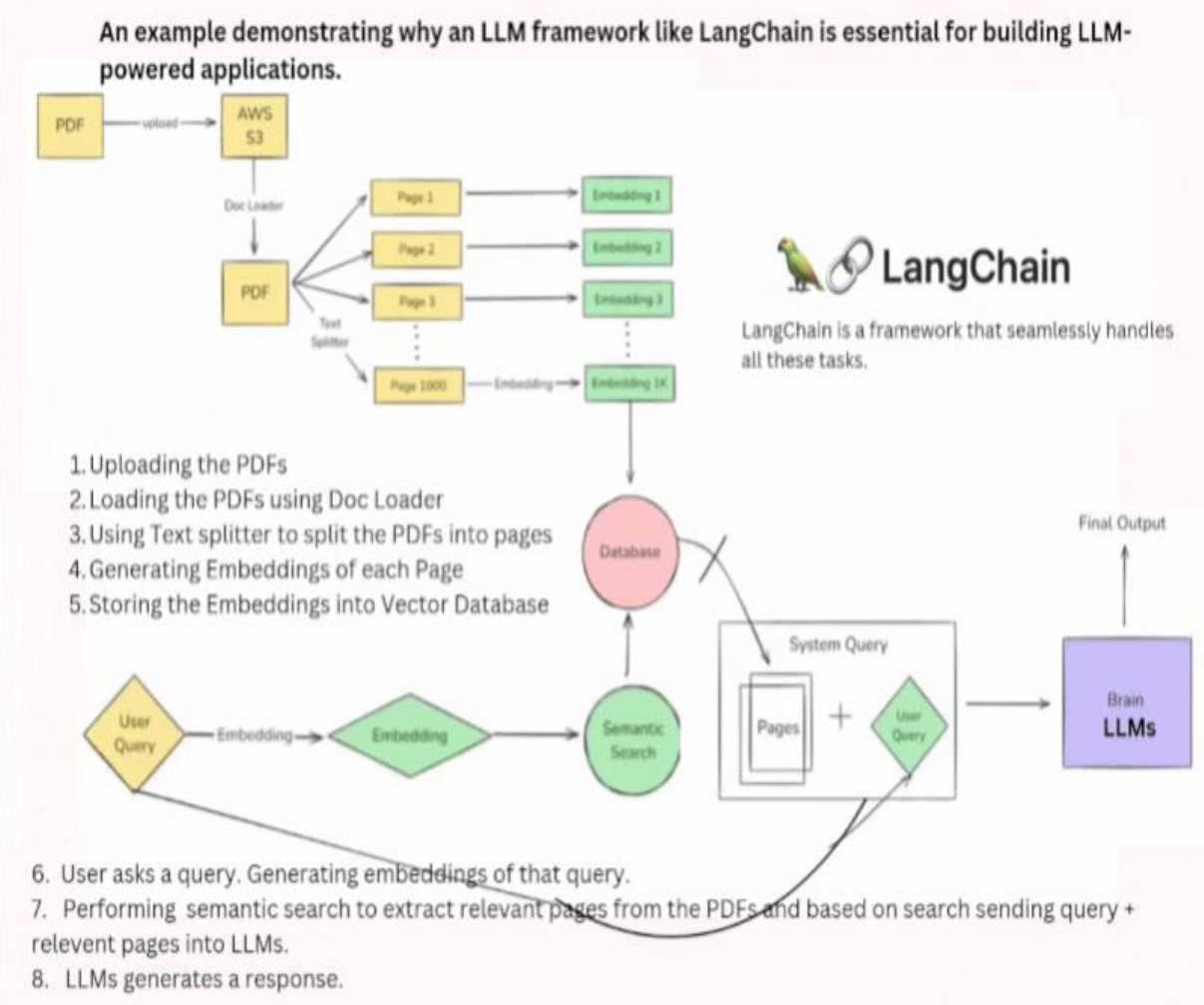
To make this work, you’d need:

1. Data Storage & Retrieval— Efficiently store and fetch documents.

2. Semantic Search— Understand user intent beyond keyword matching.

3. LLM Integration— Connect with powerful models like GPT, LLaMA, or Claude.

4. Pipeline Management— Ensure document processing, embeddings, and search indexing work smoothly.



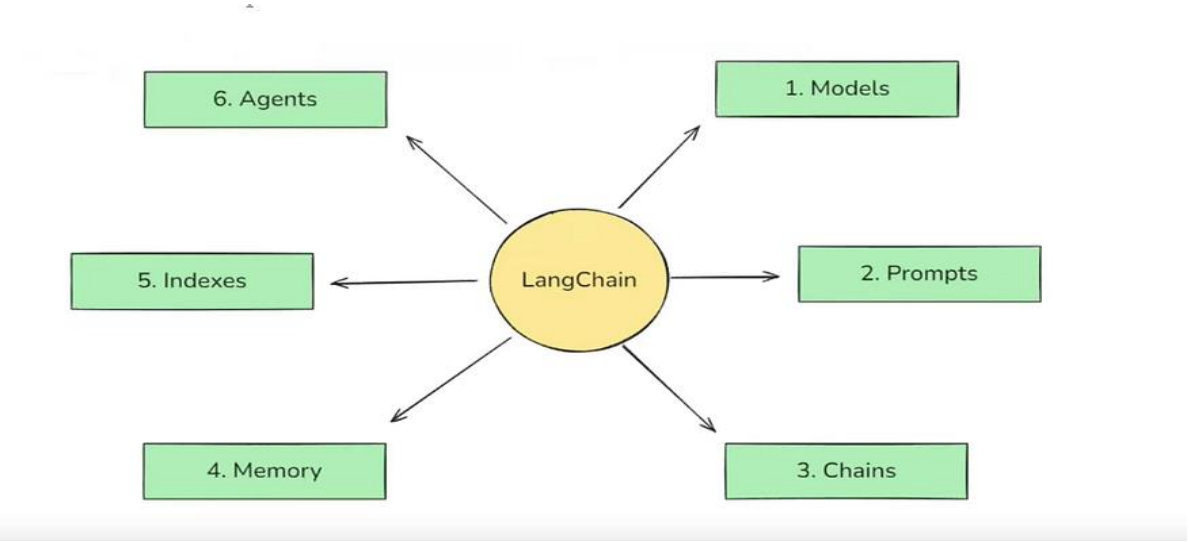
Enter LangChain — the framework that abstracts these complexities and provides:

- Pre-built modules for document processing and semantic search.
- Seamless LLM orchestration for better responses.
- Memory management for maintaining conversation context.
- Easy integration with external tools for enhanced functionality.

With LangChain, you don't have to reinvent the wheel. It empowers developers to focus on innovation rather than re-solving foundational problems.

Langchain Components :

There are 6 components in langchain, If you understand the 6 core components of langchain, you'll have a solid grasp of how to use the framework effectively.



1. Models

Let's begin with the first component: **Models**. The models component is perhaps the most important in LangChain because it serves as the core interface through which you interact with AI models. To understand this, let me provide a bit of background.

In the history of Natural Language Processing (NLP), one of the most sought-after applications was building chatbots. Chatbots have always been one of the most popular applications in NLP. However, building an effective chatbot came with two major challenges. The first was understanding the user's query. For example, if I typed "Hi, can you check my email now?" the chatbot had to figure out exactly what I meant. This challenge is known as **Natural Language Understanding (NLU)**.

The second challenge was even more complex: once the chatbot understood the query, it had to generate an appropriate response. This involves **context-aware text generation**, which was another significant hurdle.

Both of these challenges were eventually addressed with the advent of **Large Language Models (LLMs).** LLMs solved these issues by not only developing an understanding of natural language but also acquiring the ability to generate contextually appropriate responses. This breakthrough helped solve the two major problems NLP faced: understanding user queries and generating meaningful responses.

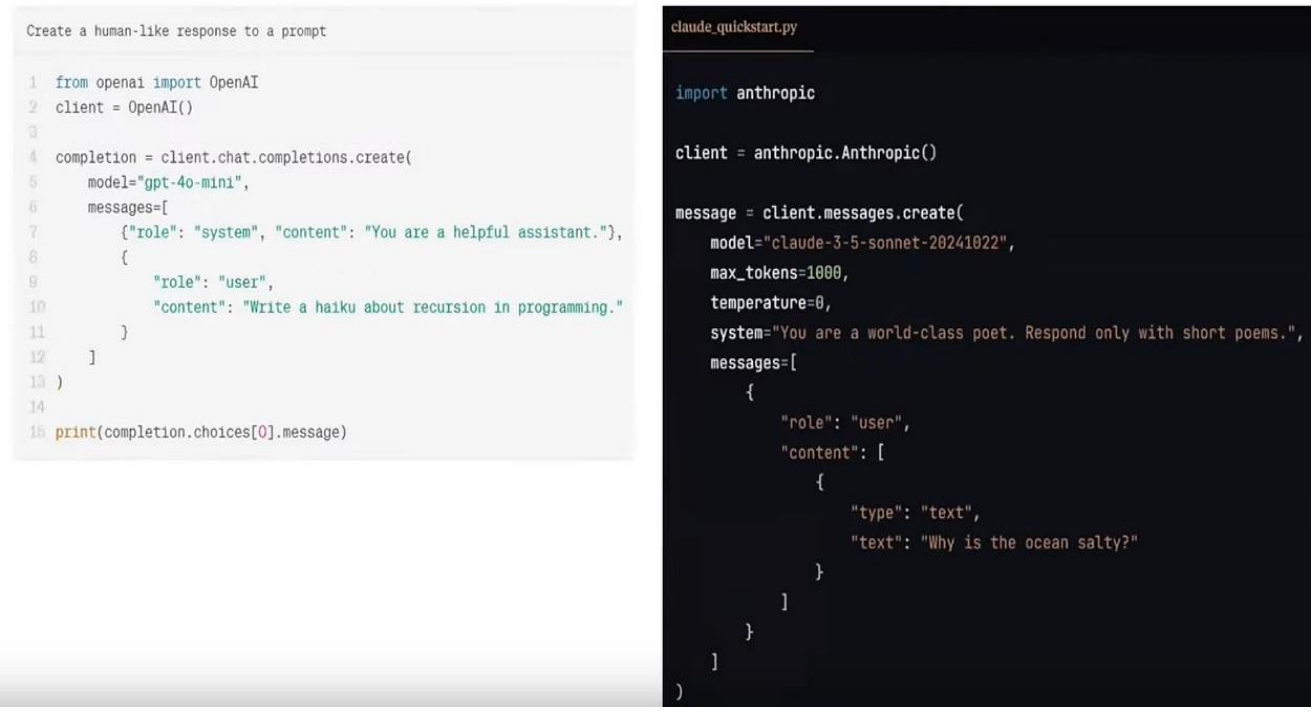
However, this advancement brought about a new challenge: the size of the models. LLMs are massive, often containing billions of parameters, making them huge files — sometimes larger than 100GB. This poses a problem because these large files cannot be

run on personal computers or smaller cloud servers due to their size and the associated costs of running them.

This problem was addressed with the introduction of **APIs**. Major companies like OpenAI created APIs, allowing anyone to send a query to the LLM via the internet. The LLM processes the request and sends back a response. This model solves the problem of managing large models because users only pay for the queries they send, without the need to run the models on their own infrastructure.

But another issue emerged: **standardization**. Different LLM providers offer their APIs in different formats, which means that as a developer, if you need to interact with multiple APIs in a single application, you'd have to write different pieces of code for each one.

For example, consider two APIs: one from OpenAI and another from a company like Anthropic. You'd have to write separate code to communicate with each API. Even when switching between models from the same provider, like switching from OpenAI to a cheaper option, you'd need to rewrite significant parts of your code.



This is where LangChain steps in. LangChain's Models component solves this issue by providing a standardized interface. With LangChain, you can easily switch between APIs with minimal changes in your code. Whether you're working with OpenAI's API or another provider like Anthropic, LangChain ensures that the interface remains the same, making it easier to integrate different models into your application.

Here's an example from LangChain code that shows how easy it is to switch between APIs:



```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(model='gpt-4', temperature=0)

result = model.invoke("Now divide the result by 1.5")

print(result.content)
```



```python
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv

load_dotenv()

model = ChatAnthropic(model='claude-3-opus-20240229')

result = model.invoke("Hi who are you")

print(result.content)
```

As you can see, the code structure remains nearly identical. The only difference is the package being imported and a few lines of configuration. This makes it incredibly easy to switch between APIs without changing your application's codebase significantly.

In short, the Models component in LangChain standardizes the interface for communicating with various AI models, making it easier to integrate and switch between them.

Now, one more thing I'll discuss is that in LangChain, you can communicate with two types of AI models:

1. **Language Model**
2. **Embedding Model.**

So, Language Models are basically LLMs, where you provide them with a text input, like "How are you today?", and they will respond with a text output like "I'm good, how about you?".
On the other hand, **Embedding Models** are slightly different. These models also take text as input, but they return a vector as output. The main use case for embedding models is **semantic search**.
In summary, the model component in LangChain is basically an interface that allows you to communicate with AI models. The big problem it solves is that previously, each LLM provider had its own API and code, but this component standardizes that, so you can talk to any LLM provider by just changing a little bit of code. That's the whole idea behind this component.

2. Prompts :

Now, let's understand what prompts are. So, when you're working with an LLM, the input you send to the LLM is called a prompt. Prompts are basically the inputs provided

to an LLM. For example, if you're working with ChatGPT and you ask the question, "What is LLMs?", the string "What is LLMs?" is considered a **prompt**.

Prompts are extremely important in the world of LLMs because the output of an LLM heavily depends on the prompt you provide. In fact, the output is very sensitive to the prompt. For instance, if you asked an LLM, "Explain Machine Learning in an academic tone," and then asked it the same question with a slight change, like "Explain Machine Learning in a fun tone," you'd notice that the output changes drastically even though you only changed one word.

LangChain has also recognized this, and since LLMs are heavily dependent on prompts, LangChain provides an excellent component to handle prompts. It offers a lot of flexibility in creating dynamic, reusable, and powerful prompts.

For example, with LangChain, you can create dynamic prompts where you don't know in advance what topic the user will ask about or in what tone. You could create a prompt like, "Summarize this topic in this tone," with placeholders for the topic and tone. Then, you can quickly replace the placeholders based on what the user wants, like asking for a fun summary of cricket or a serious summary of biology.

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template('Summarize {topic} in {emotion} tone')

print(prompt.format(topic='Cricket', length='fun'))
```

Another example is role-based prompts, where you can guide the LLM to behave like a specific role. For instance, you could tell the LLM, "You are an experienced doctor. Tell me about viral fever," and in the future, another user could ask, "You are an experienced engineer. Tell me about bridge construction."

```
# Define the ChatPromptTemplate using from_template
chat_prompt = ChatPromptTemplate.from_template([
    ("system", "Hi you are a experienced {profession}"),
    ("user", "Tell me about {topic}"),
])

# Format the prompt with the variable
formatted_messages = chat_prompt.format_messages(profession="Doctor", topic="Viral Fever")
```

You can also create few-shot prompts, where you show the LLM some examples first and then ask it a question. For example, if you're building a chatbot for customer support, you can show it examples like "I was charged twice for my subscription this month" (billing issue) and "The app crashes every time I try to log in" (technical issue). Once the LLM learns from these examples, you can ask it to categorize new queries, like "Can you explain how to upgrade my plan?"

```
examples = [
    {"input": "I was charged twice for my subscription this month.", "output": "Billing Issue"},
    {"input": "The app crashes every time I try to log in.", "output": "Technical Problem"},
    {"input": "Can you explain how to upgrade my plan?", "output": "General Inquiry"},
    {"input": "I need a refund for a payment I didn't authorize.", "output": "Billing Issue"},
]
```

```
# Step 2: Create an example template
example_template = """
Ticket: {input}
Category: {output}
"""
```

```
# Step 3: Build the few-shot prompt template
few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=PromptTemplate(input_variables=["input", "output"], template=example_template),
    prefix="Classify the following customer support tickets into one of the categories: 'Billing
    Issue', 'Technical Problem', or 'General Inquiry'.\n\n",
    suffix="\nTicket: {user_input}\nCategory:",
    input_variables=["user_input"],
)
```

```
Classify the following customer support tickets into one of the categories: 'Billing Issue', 'Technical Problem', or 'General
Inquiry'.

Ticket: I was charged twice for my subscription this month.
Category: Billing Issue

Ticket: The app crashes every time I try to log in.
Category: Technical Problem

Ticket: Can you explain how to upgrade my plan?
Category: General Inquiry

Ticket: I need a refund for a payment I didn't authorize.
Category: Billing Issue

Ticket: I am unable to connect to the internet using your service.
Category:
```

In LangChain, you can easily create these types of prompts and handle them in a structured way. Though the code may seem a bit complex if you're not familiar with it, the key takeaway is that LangChain offers a wide range of prompt engineering techniques that you can implement to work efficiently with LLMs.

3. Chains

Chains are essentially components that help us build pipelines within LangChain. The core idea is simple: whenever you create a large language model (LLM) application, you can structure it as a pipeline. And using Chains, you can build and manage that pipeline efficiently. For example:
Imagine you want to create an LLM-based application where a user gives a long English paragraph — let's say around 1000 words — and your application returns a **summary in Hindi** with fewer than 100 words.

To build this, you decide on the following flow:

1. The user provides an English input.
2. This input is first sent to an LLM that translates it into Hindi.
3. The translated Hindi text is then passed to a second LLM, which **generates a summary** in Hindi.

So now, you've got your application logic figured out. Without Chains, you'd have to manually handle this pipeline — take the user's input, pass it to the first LLM, get the translation, then take that result and feed it to the second LLM for summarization, and finally return the output to the user.
That's a lot of manual orchestration!

Enter Chains

Chains handle this orchestration automatically. The biggest advantage of using Chains is that the output of one stage automatically becomes the input for the next.

With Chains, all you need to do is feed the initial input (the English paragraph) into the Chain, and behind the scenes, everything is taken care of — translation, summarization, and final output. You don't need to worry about manually moving data from one step to the next.
In simple terms, Chains let you build pipelines in LangChain where each step flows into the next effortlessly.

4. Indexes :

ChatGPT is trained on a large corpus of internet data, and hence it can answer general queries like:
"Who is the Prime Minister of India?"
But what if you ask:
"What is the leave policy of my company XYZ?"
Well... it won't be able to answer, because that information wasn't in its training data. It's private and internal to your organization.

Here's where LangChain Indexes come in — they help you connect your LLM to private, external knowledge sources, so it can answer domain-specific queries like company policies, internal documentation, etc.

The 4 Key Components of an Index :

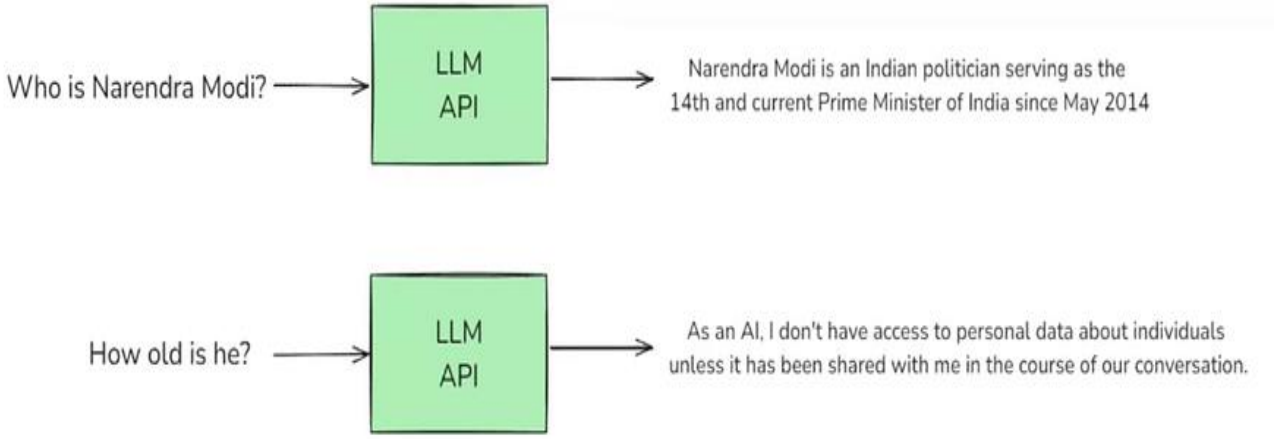
1. Document Loader
   This is responsible for loading your data — whether it's a PDF on Google Drive, a webpage, or a database.
2. Text Splitter
   Since documents can be large (say, a 1000-page rulebook), we need to break them into smaller chunks for semantic processing — often by page, paragraph, or chapter.
3. Vector Store
   Each chunk of text is embedded using a model (like OpenAI or HuggingFace embeddings), and stored in a vector database for fast similarity search.
4. Retriever
   When a user asks a query, the retriever embeds that query, performs a semantic search on the vector store, fetches relevant chunks, and sends them to the LLM for generating a final answer.

5. Memory :

“LLM API calls are stateless.”

When you start building LLM-based applications, you’ll quickly realize this core limitation: every API call you make to an LLM is stateless.

What does that mean?
Let’s understand with a simple example.
Imagine I’m using an LLM API — say, OpenAI’s GPT API. I send a prompt:
“Who is Narendra Modi?”
The model immediately responds:
“Narendra Modi is an Indian politician who is the current Prime Minister of India.”
Cool. Now, I change the next query in my code to:
“How old is he?”



This is clearly a follow-up to the previous question. But when I send this to the same model, guess what it replies?
“As an AI, I don’t have access to personal data about individuals unless it has been shared with me in the conversation.”
Wait, what? It doesn’t even remember that the previous question was about Narendra Modi!
And that is the meaning of stateless. Every API call is independent — the model has no memory of what came before. And this becomes a huge problem when you’re trying to build chatbots or any conversational agent.

Without memory, every turn in the conversation feels disconnected. The user would need to remind the bot every single time about the context — which is frustrating.

LangChain’s Memory Component

This exact problem is what LangChain addresses through its Memory component.

Using memory, you can inject contextual continuity into your conversation with the LLM. LangChain provides different types of memory depending on your use case. Let's look at the commonly used ones:

1. **ConversationBufferMemory**

This one stores all the messages exchanged so far. So when you make the next API call, you pass the entire chat history along with it. That way, the model knows what has been discussed so far and responds accordingly.

However, there's a drawback — if the conversation gets too long, the chat history becomes huge. And that increases the cost, since more tokens = more money.

2. **ConversationBufferWindowMemory**

Instead of storing all the conversation history, this stores only the last N interactions (say, the last 100 messages). It's a sliding window that constantly updates and keeps only the recent history.
This helps in reducing token usage while still preserving some level of context.

3. **ConversationSummaryMemory**

Here, instead of sending full conversation history, you generate a summary of the previous messages and send that along with your next prompt. This helps preserve context while using significantly fewer tokens.

4. **CustomMemory**

This is for more advanced use cases. Here, you can store specific user preferences, facts, or figures that you want to persist across conversations. For example, if the user's name, likes, or purchase history are important to the chat, you store them in memory explicitly.

6. Agents :

Now let's talk about the final and arguably the most exciting component in the LangChain ecosystem — **Agents**.
Let me explain this with a simple example.
Suppose you're on a travel website like MakeMyTrip, and you ask the chatbot:
"Can you tell me the best travel destinations in India during summer?"
Since this chatbot is powered by an LLM trained on internet data, it might reply:
"You can visit Shimla or Manali. Hill stations are great in summer."
Now imagine if instead of a chatbot, you were talking to an AI agent.
Then, you could go one step further and ask:
"Can you tell me the cheapest flight from Delhi to Shimla on January 24th?"

What will the AI agent do?

It will instantly call a flight API, fetch the latest flight data, and respond:

“The cheapest flight is from Indigo, departing on January 24th from Delhi to Shimla.”
And guess what? You can then say:
“Can you book that flight for me?”
And the agent will go to the MakeMyTrip website and book the flight for you. That’s the power of agents.

Chatbots vs Agents

Let me make it super simple:

- A chatbot can talk to you.
- An agent can talk to you and perform actions on your behalf.

What Makes an Agent Special?

There are two things that agents have, which normal chatbots don’t:

1. Reasoning Capability
2. Access to Tools (like APIs, calculators, search engines, etc.)

These two capabilities are what make agents so powerful.
Let’s see how these work together.

How Agents Work — Step by Step
Imagine we build a basic AI agent and give it two tools:
A Calculator
Access to a Weather API
Now a user comes in and asks:
“Can you multiply today’s temperature in Delhi by 3?”

Here’s how the agent processes this:

- First, because it has reasoning capability, it will break the query into steps using a technique called Chain of Thought prompting.
- Step 1: I need today’s temperature in Delhi.
- Step 2: Once I have it, I need to multiply it by 3.

So the agent focuses on step 1 and checks if it has a tool that can get the temperature.
It notices: “Yes! I have access to a weather API.”
So it calls the API with input Delhi, and let’s say it gets:
25°C
Now it moves to step 2:
Multiply 25 by 3.
To do this, it needs a calculator. It checks its tools again: “Yes, I have one!”
So it calls the calculator with input 25 * 3.

And finally, it gets:
75
That becomes the final output — and that's how an AI agent completes a task.

Summary

In this first part of the "Mastering LangChain" series, we explored the foundational components of LangChain — a powerful framework for building LLM-based applications. We covered the key elements: Models, Prompts, Chains, Indexes, Memory and Agents, understanding how each contributes to creating smarter, more structured, and efficient language model applications. From standardizing API calls across LLMs to building robust pipelines and enabling domain-specific retrieval, LangChain offers a modular approach to unlocking the full potential of LLMs.